Nand2Tetris

Project 1 (Boolean Logic) & Project 2 (Boolean Arithmetic) — HDL Solutions

NAME	ROLL NO.	COURSE	SECTION	SUBJECT
Yash Dagar	**2501730474**	**B.Tech CSE (AI&ML)**	**G**	**Nand2Tetris**

CONTENTS

Yash Dagar

#	CHIP	PROJ.	WHAT IT DOES
1	**And**	P1	out = 1 only when both a and b are 1. And = Not(Nand).
2	**Not**	P1	Tie both Nand inputs together: Nand(in, in) = Not(in).
3	**Or**	P1	De Morgan: a Or b = Nand(Not a, Not b).
4	**Xor**	P1	out = 1 when a and b differ. Built from 4 Nand gates.
5	**Mux**	P1	2-way multiplexor: sel = 0 selects a, sel = 1 selects b.
6	**DMux**	P1	Demultiplexor: sends in to a (sel = 0) or to b (sel = 1); the other output is 0.
7	**Not16**	P1	Applies Not to each of the 16 bits independently.
8	**And16**	P1	Bitwise And of two 16-bit buses.
9	**Or16**	P1	Bitwise Or of two 16-bit buses.
10	**Mux16**	P1	16-bit multiplexor: sel = 0 gives a, sel = 1 gives b (one Mux per bit).
11	**Or8Way**	P1	out = in[0] Or in[1] Or ... Or in[7], chained one Or at a time.
12	**Mux4Way16**	P1	Selects one of a, b, c, d: sel[0] picks within each pair, sel[1] picks the pair.
13	**Mux8Way16**	P1	Selects one of a..h: two Mux4Way16 on sel[0..1], then Mux16 on sel[2].
14	**DMux4Way**	P1	Routes in to one of a, b, c, d: sel[1] picks the half, sel[0] picks within it.
15	**DMux8Way**	P1	Routes in to one of a..h: DMux on sel[2], then two DMux4Way on sel[0..1].
16	**HalfAdder**	P2	Adds two bits: sum = a Xor b, carry = a And b.
17	**FullAdder**	P2	Adds three bits using two HalfAdders; carry = c1 Or c2.
18	**Add16**	P2	Ripple-carry adder: HalfAdder for bit 0, FullAdder for bits 1-15; final carry is ignored.
19	**Inc16**	P2	out = in + 1: add the constant 0000000000000001 using Add16.
20	**ALU**	P2	Pre-process x and y (zero / negate), compute x + y or x & y, optionally negate, then set zr and ng flags.

01 And.hdl

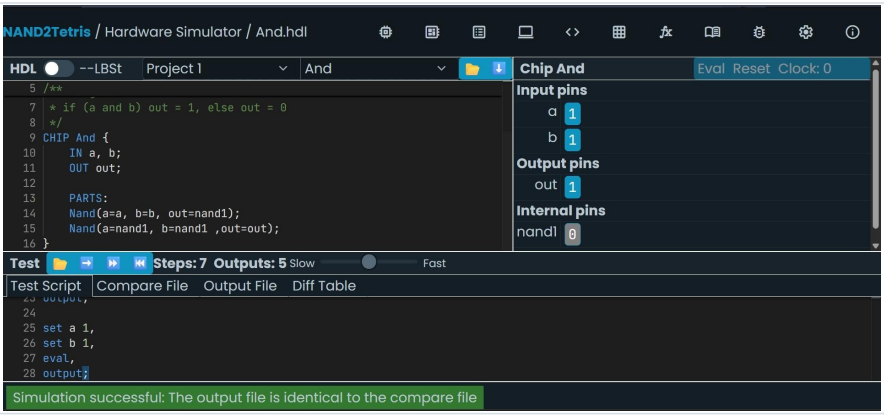
PROJECT 1

out = 1 only when both a and b are 1. And = Not(Nand).

CODE

```
CHIP And {
    IN a, b;
    OUT out;
    PARTS:
    Nand(a=a, b=b, out=nand1);
    Nand(a=nand1, b=nand1, out=out);
}
```

HARDWARE SIMULATOR OUTPUT



02 Not.hdl

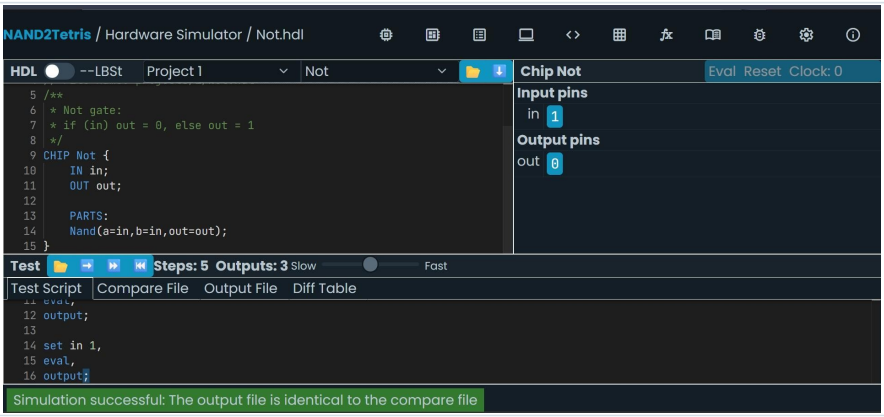
PROJECT 1

Tie both Nand inputs together: Nand(in, in) = Not(in).

CODE

```
CHIP Not {
    IN in;
    OUT out;
    PARTS:
    Nand(a=in, b=in, out=out);
}
```

HARDWARE SIMULATOR OUTPUT



03 Or.hdl

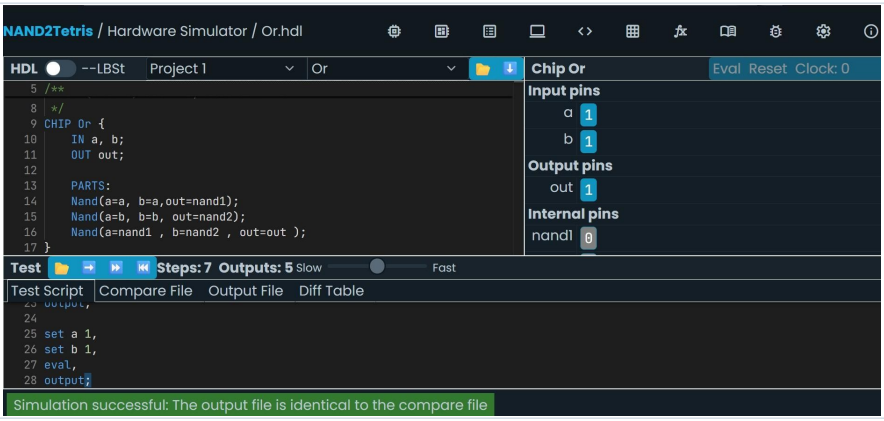
PROJECT 1

De Morgan: a Or b = Nand(Not a, Not b).

CODE

```
CHIP Or {
    IN a, b;
    OUT out;
    PARTS:
    Nand(a=a, b=a, out=nota);
    Nand(a=b, b=b, out=notb);
    Nand(a=nota, b=notb, out=out);
}
```

HARDWARE SIMULATOR OUTPUT



04 Xor.hdl

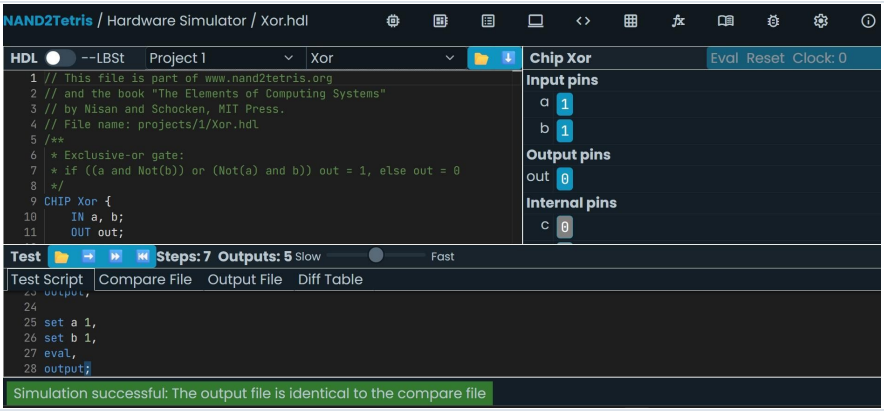
PROJECT 1

out = 1 when a and b differ. Built from 4 Nand gates.

CODE

```
CHIP Xor {
    IN a, b;
    OUT out;
    PARTS:
    Nand(a=a, b=b, out=c);
    Nand(a=a, b=c, out=d);
    Nand(a=c, b=b, out=e);
    Nand(a=d, b=e, out=out);
}
```

HARDWARE SIMULATOR OUTPUT



05 Mux.hdl

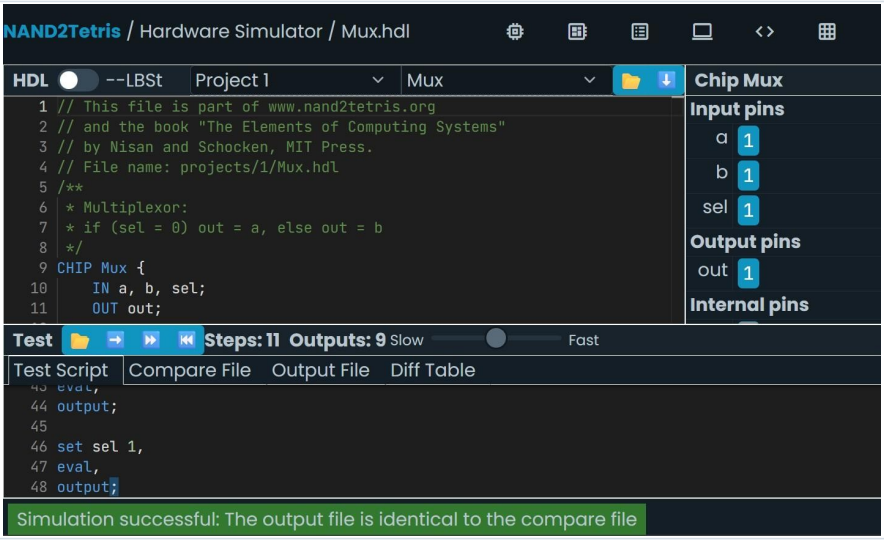
PROJECT 1

2-way multiplexor: sel = 0 selects a, sel = 1 selects b.

CODE

```
CHIP Mux {
    IN a, b, sel;
    OUT out;
    PARTS:
    Not(in=sel, out=nsel);
    And(a=a, b=nsel, out=a1);
    And(a=b, b=sel, out=b1);
    Or(a=a1, b=b1, out=out);
}
```

HARDWARE SIMULATOR OUTPUT



06 DMux.hdl

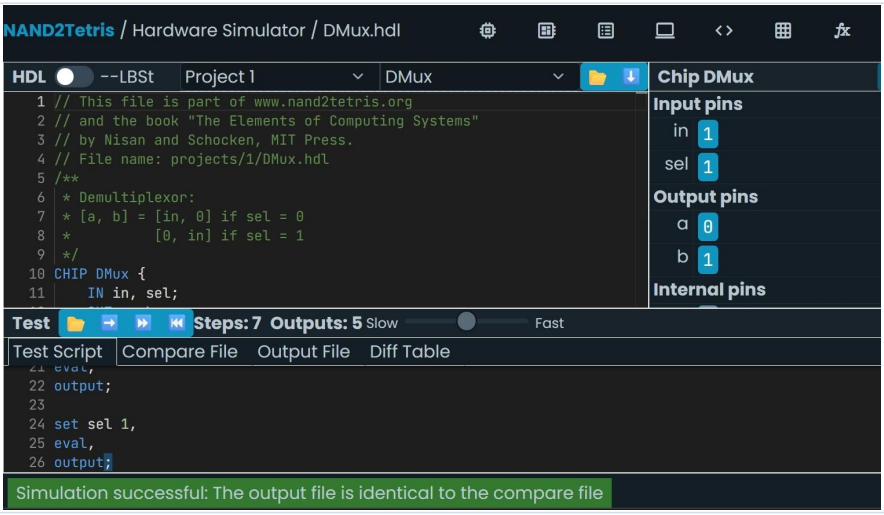
PROJECT 1

Demultiplexor: sends in to a (sel = 0) or to b (sel = 1); the other output is 0.

CODE

```
CHIP DMux {
    IN in, sel;
    OUT a, b;
    PARTS:
    Not(in=sel, out=nsel);
    And(a=in, b=nsel, out=a);
    And(a=in, b=sel, out=b);
}
```

HARDWARE SIMULATOR OUTPUT



07 Not16.hdl

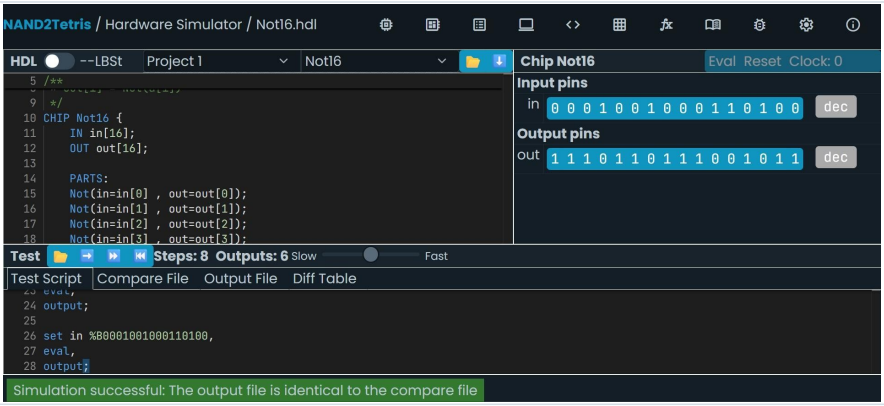
PROJECT 1

Applies Not to each of the 16 bits independently.

CODE

```
CHIP Not16 {
    IN in[16];
    OUT out[16];
    PARTS:
    Not(in=in[0], out=out[0]);
    Not(in=in[1], out=out[1]);
    Not(in=in[2], out=out[2]);
    Not(in=in[3], out=out[3]);
    Not(in=in[4], out=out[4]);
    Not(in=in[5], out=out[5]);
    Not(in=in[6], out=out[6]);
    Not(in=in[7], out=out[7]);
    Not(in=in[8], out=out[8]);
    Not(in=in[9], out=out[9]);
    Not(in=in[10], out=out[10]);
    Not(in=in[11], out=out[11]);
    Not(in=in[12], out=out[12]);
    Not(in=in[13], out=out[13]);
    Not(in=in[14], out=out[14]);
    Not(in=in[15], out=out[15]);
}
```

HARDWARE SIMULATOR OUTPUT



08 And16.hdl

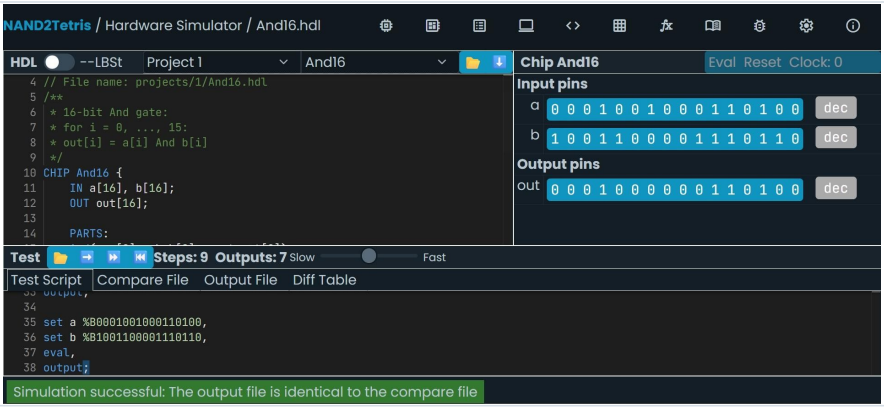
PROJECT 1

Bitwise And of two 16-bit buses.

CODE

```
CHIP And16 {
    IN a[16], b[16];
    OUT out[16];
    PARTS:
    And(a=a[0], b=b[0], out=out[0]);
    And(a=a[1], b=b[1], out=out[1]);
    And(a=a[2], b=b[2], out=out[2]);
    And(a=a[3], b=b[3], out=out[3]);
    And(a=a[4], b=b[4], out=out[4]);
    And(a=a[5], b=b[5], out=out[5]);
    And(a=a[6], b=b[6], out=out[6]);
    And(a=a[7], b=b[7], out=out[7]);
    And(a=a[8], b=b[8], out=out[8]);
    And(a=a[9], b=b[9], out=out[9]);
    And(a=a[10], b=b[10], out=out[10]);
    And(a=a[11], b=b[11], out=out[11]);
    And(a=a[12], b=b[12], out=out[12]);
    And(a=a[13], b=b[13], out=out[13]);
    And(a=a[14], b=b[14], out=out[14]);
    And(a=a[15], b=b[15], out=out[15]);
}
```

HARDWARE SIMULATOR OUTPUT



09 Or16.hdl

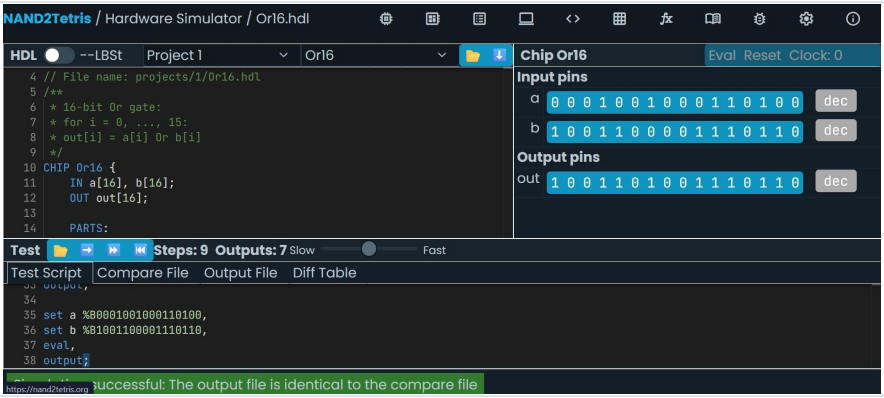
PROJECT 1

Bitwise Or of two 16-bit buses.

CODE

```
CHIP Or16 {
    IN a[16], b[16];
    OUT out[16];
    PARTS:
    Or(a=a[0], b=b[0], out=out[0]);
    Or(a=a[1], b=b[1], out=out[1]);
    Or(a=a[2], b=b[2], out=out[2]);
    Or(a=a[3], b=b[3], out=out[3]);
    Or(a=a[4], b=b[4], out=out[4]);
    Or(a=a[5], b=b[5], out=out[5]);
    Or(a=a[6], b=b[6], out=out[6]);
    Or(a=a[7], b=b[7], out=out[7]);
    Or(a=a[8], b=b[8], out=out[8]);
    Or(a=a[9], b=b[9], out=out[9]);
    Or(a=a[10], b=b[10], out=out[10]);
    Or(a=a[11], b=b[11], out=out[11]);
    Or(a=a[12], b=b[12], out=out[12]);
    Or(a=a[13], b=b[13], out=out[13]);
    Or(a=a[14], b=b[14], out=out[14]);
    Or(a=a[15], b=b[15], out=out[15]);
}
```

HARDWARE SIMULATOR OUTPUT



10 Mux16.hdl

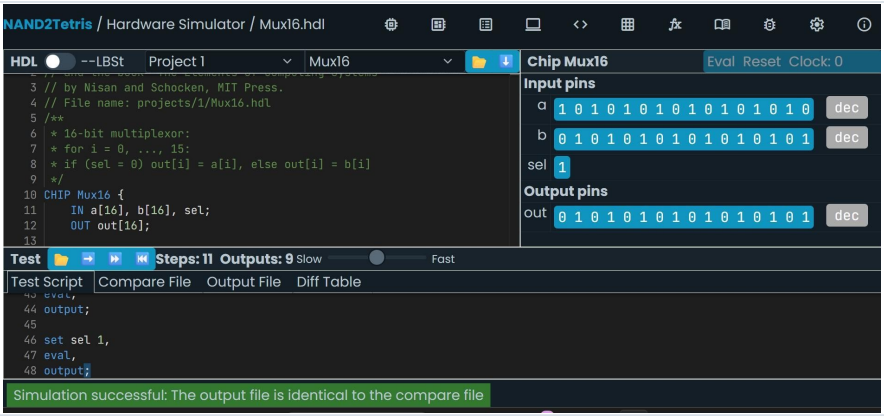
PROJECT 1

16-bit multiplexor: sel = 0 gives a, sel = 1 gives b (one Mux per bit).

CODE

```
CHIP Mux16 {
    IN a[16], b[16], sel;
    OUT out[16];
    PARTS:
    Mux(a=a[0], b=b[0], sel=sel, out=out[0]);
    Mux(a=a[1], b=b[1], sel=sel, out=out[1]);
    Mux(a=a[2], b=b[2], sel=sel, out=out[2]);
    Mux(a=a[3], b=b[3], sel=sel, out=out[3]);
    Mux(a=a[4], b=b[4], sel=sel, out=out[4]);
    Mux(a=a[5], b=b[5], sel=sel, out=out[5]);
    Mux(a=a[6], b=b[6], sel=sel, out=out[6]);
    Mux(a=a[7], b=b[7], sel=sel, out=out[7]);
    Mux(a=a[8], b=b[8], sel=sel, out=out[8]);
    Mux(a=a[9], b=b[9], sel=sel, out=out[9]);
    Mux(a=a[10], b=b[10], sel=sel, out=out[10]);
    Mux(a=a[11], b=b[11], sel=sel, out=out[11]);
    Mux(a=a[12], b=b[12], sel=sel, out=out[12]);
    Mux(a=a[13], b=b[13], sel=sel, out=out[13]);
    Mux(a=a[14], b=b[14], sel=sel, out=out[14]);
    Mux(a=a[15], b=b[15], sel=sel, out=out[15]);
}
```

HARDWARE SIMULATOR OUTPUT



11 Or8Way.hdl

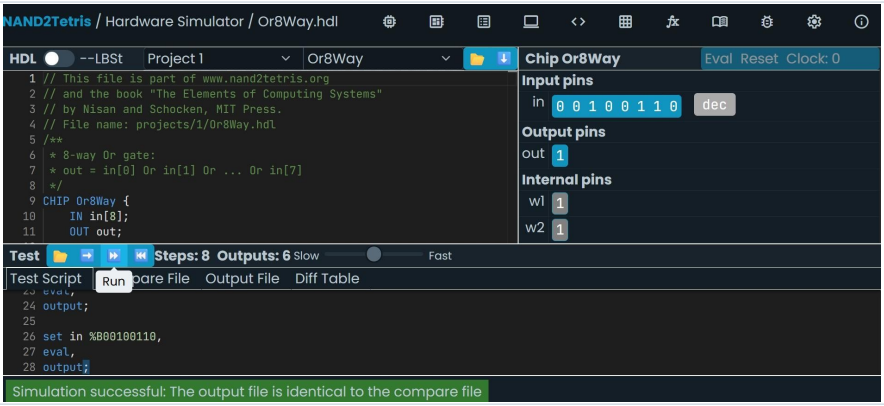
PROJECT 1

out = in[0] Or in[1] Or ... Or in[7], chained one Or at a time.

CODE

```
CHIP Or8Way {
    IN in[8];
    OUT out;
    PARTS:
    Or(a=in[0], b=in[1], out=w1);
    Or(a=w1, b=in[2], out=w2);
    Or(a=w2, b=in[3], out=w3);
    Or(a=w3, b=in[4], out=w4);
    Or(a=w4, b=in[5], out=w5);
    Or(a=w5, b=in[6], out=w6);
    Or(a=w6, b=in[7], out=out);
}
```

HARDWARE SIMULATOR OUTPUT



12 Mux4Way16.hdl

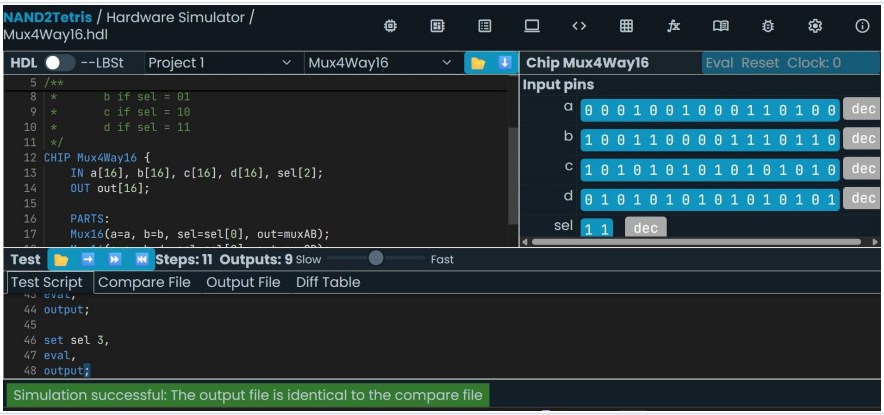
PROJECT 1

Selects one of a, b, c, d: sel[0] picks within each pair, sel[1] picks the pair.

CODE

```
CHIP Mux4Way16 {
    IN a[16], b[16], c[16], d[16], sel[2];
    OUT out[16];
    PARTS:
    Mux16(a=a, b=b, sel=sel[0], out=muxAB);
    Mux16(a=c, b=d, sel=sel[0], out=muxCD);
    Mux16(a=muxAB, b=muxCD, sel=sel[1], out=out);
}
```

HARDWARE SIMULATOR OUTPUT



13 Mux8Way16.hdl

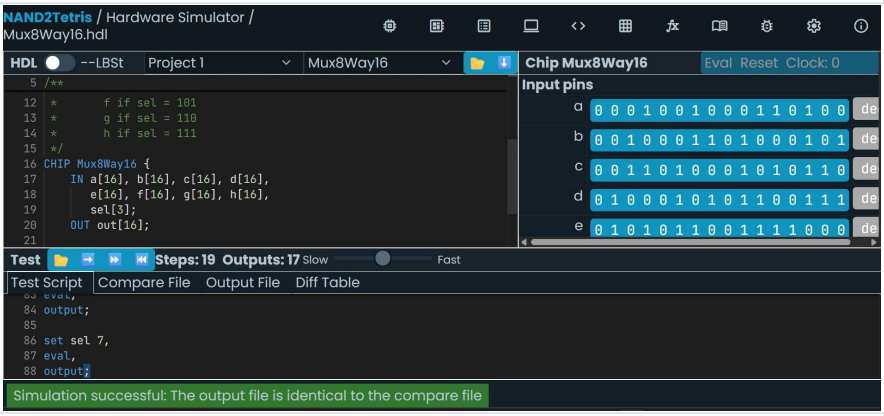
PROJECT 1

Selects one of a..h: two Mux4Way16 on sel[0..1], then Mux16 on sel[2].

CODE

```
CHIP Mux8Way16 {
    IN a[16], b[16], c[16], d[16], e[16], f[16], g[16], h[16], sel[3];
    OUT out[16];
    PARTS:
    Mux4Way16(a=a, b=b, c=c, d=d, sel=sel[0..1], out=outABCD);
    Mux4Way16(a=e, b=f, c=g, d=h, sel=sel[0..1], out=outEFGH);
    Mux16(a=outABCD, b=outEFGH, sel=sel[2], out=out);
}
```

HARDWARE SIMULATOR OUTPUT



14 DMux4Way.hdl

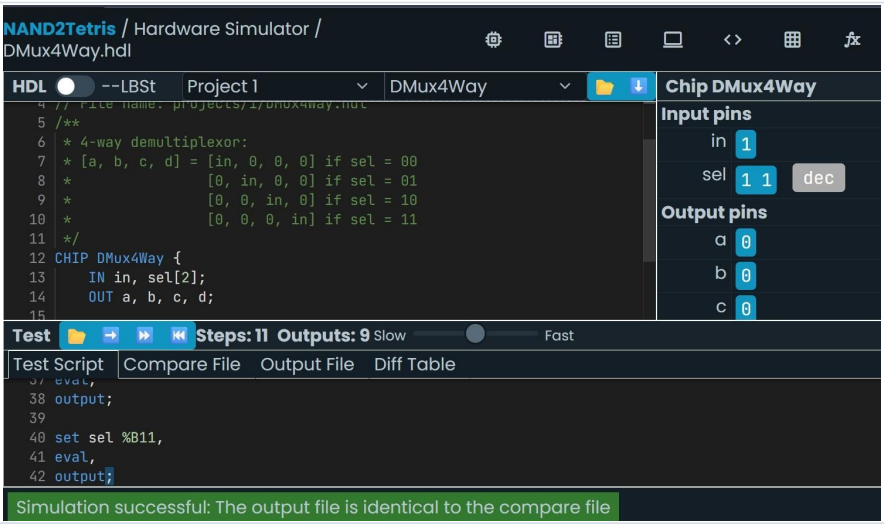
PROJECT 1

Routes in to one of a, b, c, d: sel[1] picks the half, sel[0] picks within it.

CODE

```
CHIP DMux4Way {
    IN in, sel[2];
    OUT a, b, c, d;
    PARTS:
    DMux(in=in, sel=sel[1], a=top, b=bottom);
    DMux(in=top, sel=sel[0], a=a, b=b);
    DMux(in=bottom, sel=sel[0], a=c, b=d);
}
```

HARDWARE SIMULATOR OUTPUT



15 DMux8Way.hdl

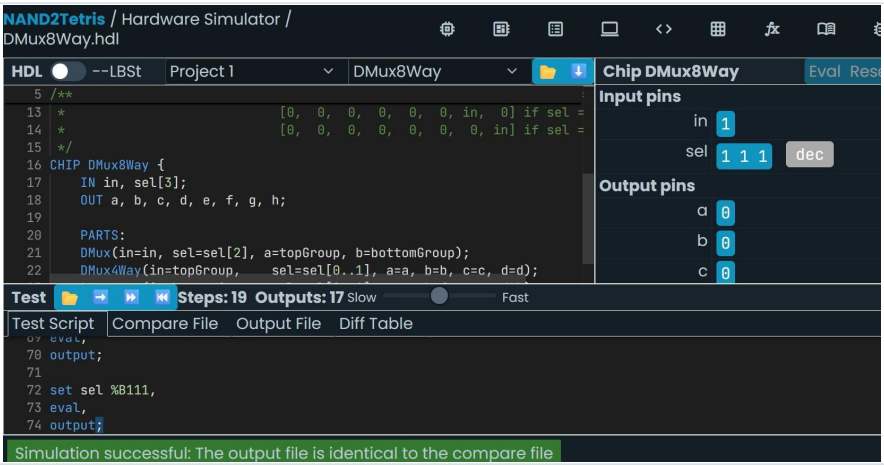
PROJECT 1

Routes in to one of a..h: DMux on sel[2], then two DMux4Way on sel[0..1].

CODE

```
CHIP DMux8Way {
    IN in, sel[3];
    OUT a, b, c, d, e, f, g, h;
    PARTS:
    DMux(in=in, sel=sel[2], a=topGroup, b=bottomGroup);
    DMux4Way(in=topGroup, sel=sel[0..1], a=a, b=b, c=c, d=d);
    DMux4Way(in=bottomGroup, sel=sel[0..1], a=e, b=f, c=g, d=h);
}
```

HARDWARE SIMULATOR OUTPUT



16 HalfAdder.hdl

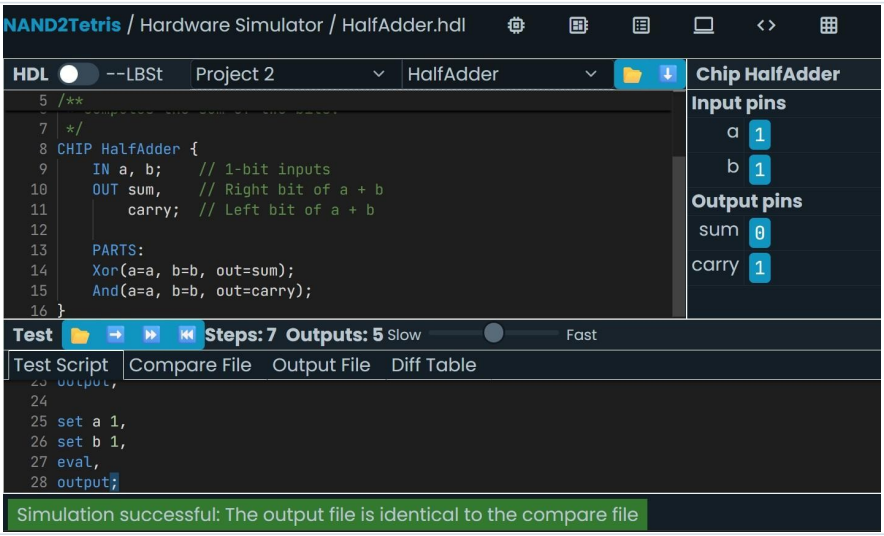
PROJECT 2

Adds two bits: sum = a Xor b, carry = a And b.

CODE

```
CHIP HalfAdder {
    IN a, b;
    OUT sum,     // Right bit of a + b
        carry;   // Left bit of a + b
    PARTS:
    Xor(a=a, b=b, out=sum);
    And(a=a, b=b, out=carry);
}
```

HARDWARE SIMULATOR OUTPUT



17 FullAdder.hdl

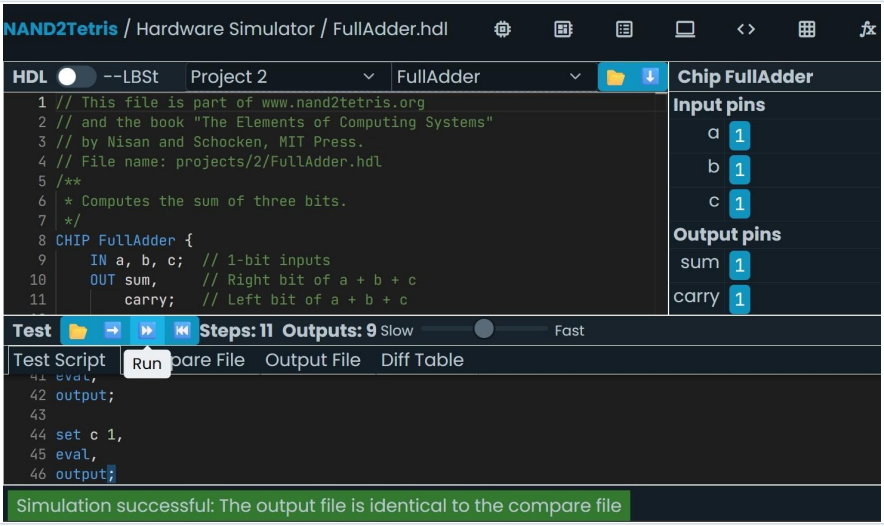
PROJECT 2

Adds three bits using two HalfAdders; carry = c1 Or c2.

CODE

```
CHIP FullAdder {
    IN a, b, c;
    OUT sum,     // Right bit of a + b + c
        carry;   // Left bit of a + b + c
    PARTS:
    HalfAdder(a=a, b=b, sum=s1, carry=c1);
    HalfAdder(a=s1, b=c, sum=sum, carry=c2);
    Or(a=c1, b=c2, out=carry);
}
```

HARDWARE SIMULATOR OUTPUT



18 Add16.hdl

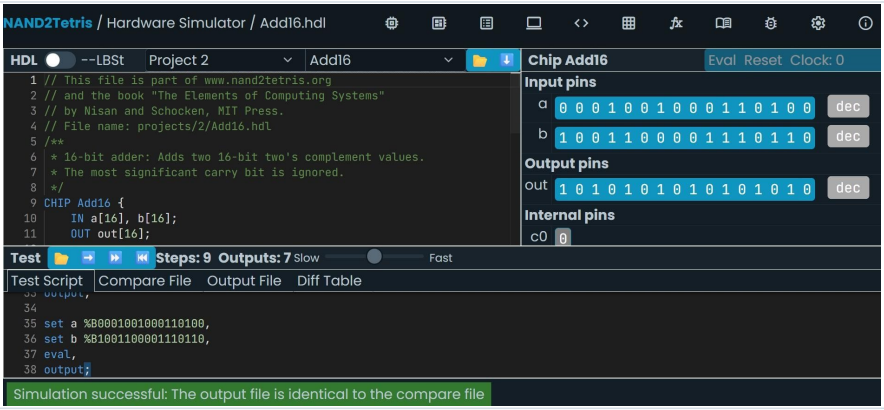
PROJECT 2

Ripple-carry adder: HalfAdder for bit 0, FullAdder for bits 1-15; final carry is ignored.

CODE

```
CHIP Add16 {
    IN a[16], b[16];
    OUT out[16];
    PARTS:
    HalfAdder(a=a[0], b=b[0], sum=out[0], carry=c0);
    FullAdder(a=a[1], b=b[1], c=c0, sum=out[1], carry=c1);
    FullAdder(a=a[2], b=b[2], c=c1, sum=out[2], carry=c2);
    FullAdder(a=a[3], b=b[3], c=c2, sum=out[3], carry=c3);
    FullAdder(a=a[4], b=b[4], c=c3, sum=out[4], carry=c4);
    FullAdder(a=a[5], b=b[5], c=c4, sum=out[5], carry=c5);
    FullAdder(a=a[6], b=b[6], c=c5, sum=out[6], carry=c6);
    FullAdder(a=a[7], b=b[7], c=c6, sum=out[7], carry=c7);
    FullAdder(a=a[8], b=b[8], c=c7, sum=out[8], carry=c8);
    FullAdder(a=a[9], b=b[9], c=c8, sum=out[9], carry=c9);
    FullAdder(a=a[10], b=b[10], c=c9, sum=out[10], carry=c10);
    FullAdder(a=a[11], b=b[11], c=c10, sum=out[11], carry=c11);
    FullAdder(a=a[12], b=b[12], c=c11, sum=out[12], carry=c12);
    FullAdder(a=a[13], b=b[13], c=c12, sum=out[13], carry=c13);
    FullAdder(a=a[14], b=b[14], c=c13, sum=out[14], carry=c14);
    FullAdder(a=a[15], b=b[15], c=c14, sum=out[15], carry=c15);
}
```

HARDWARE SIMULATOR OUTPUT



19 Inc16.hdl

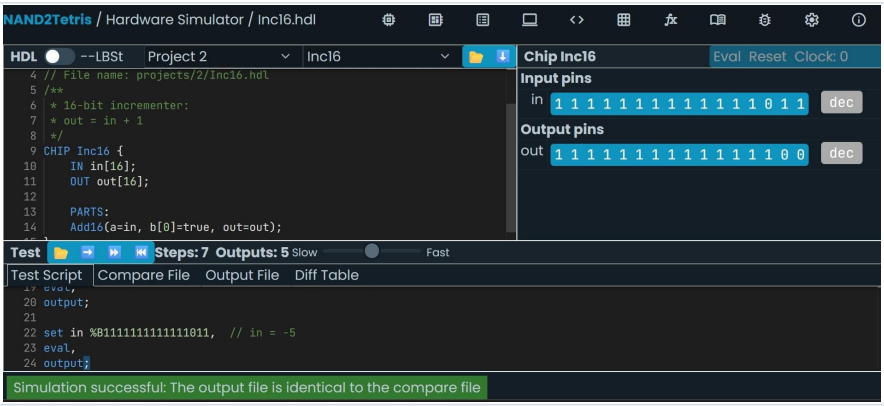
PROJECT 2

out = in + 1: add the constant 0000000000000001 using Add16.

CODE

```
CHIP Inc16 {
    IN in[16];
    OUT out[16];
    PARTS:
    Add16(a=in, b[0]=true, out=out);
}
```

HARDWARE SIMULATOR OUTPUT



20 ALU.hdl

PROJECT 2

Pre-process x and y (zero / negate), compute x + y or x & y, optionally negate, then set zr and ng flags.

CODE

```
CHIP ALU {
    IN  x[16], y[16],  // 16-bit inputs
        zx,            // zero the x input?
        nx,            // negate the x input?
        zy,            // zero the y input?
        ny,            // negate the y input?
        f,             // compute (out = x + y) or (out = x & y)?
        no;            // negate the out output?
    OUT out[16],       // 16-bit output
        zr,            // if (out == 0) equals 1, else 0
        ng;            // if (out < 0)  equals 1, else 0
    PARTS:
    // x: zero, then negate
    Mux16(a=x, sel=zx, out=x1);
    Not16(in=x1, out=negatedX);
    Mux16(a=x1, b=negatedX, sel=nx, out=x2);

    // y: zero, then negate
    Mux16(a=y, sel=zy, out=y1);
    Not16(in=y1, out=negatedY);
    Mux16(a=y1, b=negatedY, sel=ny, out=y2);

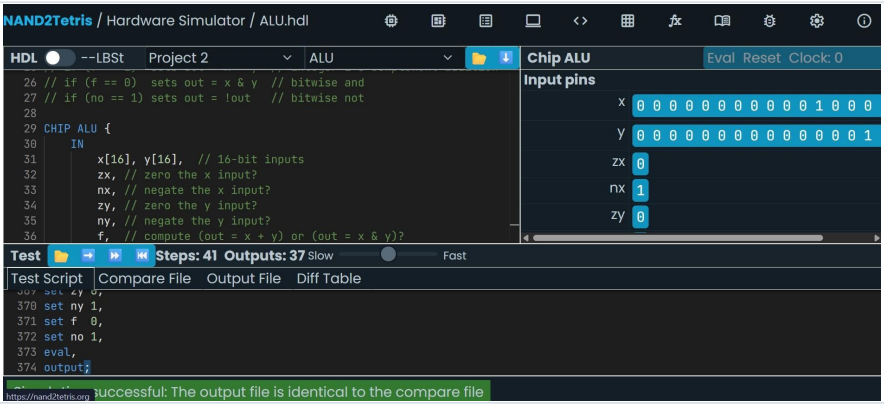
    // f: x + y or x & y
    Add16(a=x2, b=y2, out=added);
    And16(a=x2, b=y2, out=xyAnd);
    Mux16(a=xyAnd, b=added, sel=f, out=result);

    // no: negate the output
    Not16(in=result, out=negatedResult);
    Mux16(a=result, b=negatedResult, sel=no, out=out,
          out[15]=firstOut, out[0..7]=finalLeft, out[8..15]=finalRight);

    // zr: out == 0
    Or8Way(in=finalLeft, out=zrl);
    Or8Way(in=finalRight, out=zrr);
    Or(a=zrl, b=zrr, out=nzr);
    Not(in=nzr, out=zr);

    // ng: out < 0 (sign bit)
    And(a=firstOut, b=true, out=ng);
}
```

HARDWARE SIMULATOR OUTPUT



All 20 chips pass their test scripts in the Hardware Simulator — output matches the compare file.